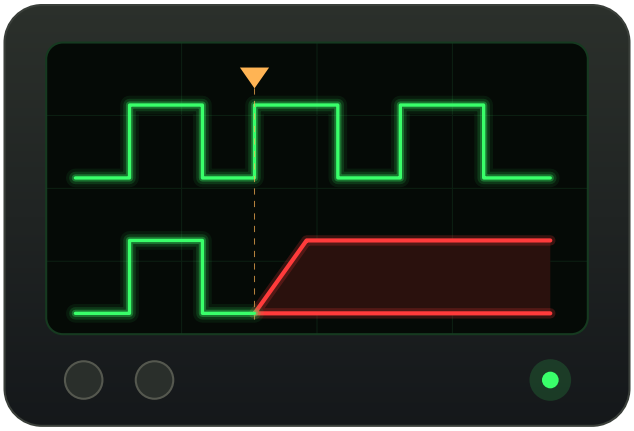

LibreILA

LibreILA User Manual

Yash Paritkar (github/yashparitkar)

August 2026

Document History

Version	Date	Comments
0.0	25 July 2026	Document created.

Contents

List of Figures

List of Tables

List of Acronyms

AXI Advanced eXtensible Interface

CDC Clock Domain Crossing

FIFO First In First Out (queue)

FPGA Field Programmable Gate Array

GUI Graphical User Interface

HDL Hardware Description Language

ILA Integrated Logic Analyzer

IP Intellectual Property

LSRAM Large Static Random Access Memory

OHL Open Hardware Licence

RTL Register Transfer Level

SDC Synopsys Design Constraints

UART Universal Asynchronous Receiver Transmitter

VCD Value Change Dump

Chapter 1

Introduction

TODO: What LibreILA is, in three paragraphs, followed by the document map.

1.1 What LibreILA is

TODO: An in-system ILA spliced into a link inside the fabric, sampling into a circular buffer in fabric RAM. Triggered, time correlated, and read back as a VCD. One short paragraph, no register detail.

1.2 Using LibreILA

TODO: The two deliverables and the chapter each one leads to: `libre_ila`, the bare core on AXI4Lite, driven by the baremetal C driver (Chapter 8), and `libre_ila_uart`, the serial wrapper, driven from a PC by the python driver (Chapters 6 and 7).

1.3 Document Map

TODO: A table of the three documents: this manual is how to use LibreILA, the datasheet is the specification, register map, timing and packet format, and the per-directory READMEs are implementation notes for someone changing the code. A user of LibreILA never needs the third.

1.4 Conventions

TODO: Shell prompts, paths relative to the repository root, register names in small caps and hex as 0x. Half a page.

Chapter 2

Getting Started

TODO: The highest value chapter in the manual: a reader who follows it from top to bottom gets a waveform on screen without opening any other document. No forward references, so anything needed here is stated here, with the full treatment cross-referenced later.

2.1 Prerequisites

The full toolchain is tested using following:

- FPGA: Microchip PolarFire SoC Discovery Kit (mpfs-disco-kit)
- Libero SoC v2025.2
- SoftConsole v2022.2
- Ubuntu 22.04.3 LTS
- Python 3.10.0
- Packages for driver: pyserial (v3.5)
- Packages for GUI: PySide6 (v6.5.1)
- Rpi Pico with `debugprobe_on_pico` firmware flashed as USB-TTL adapter (other serial adapters will also work)

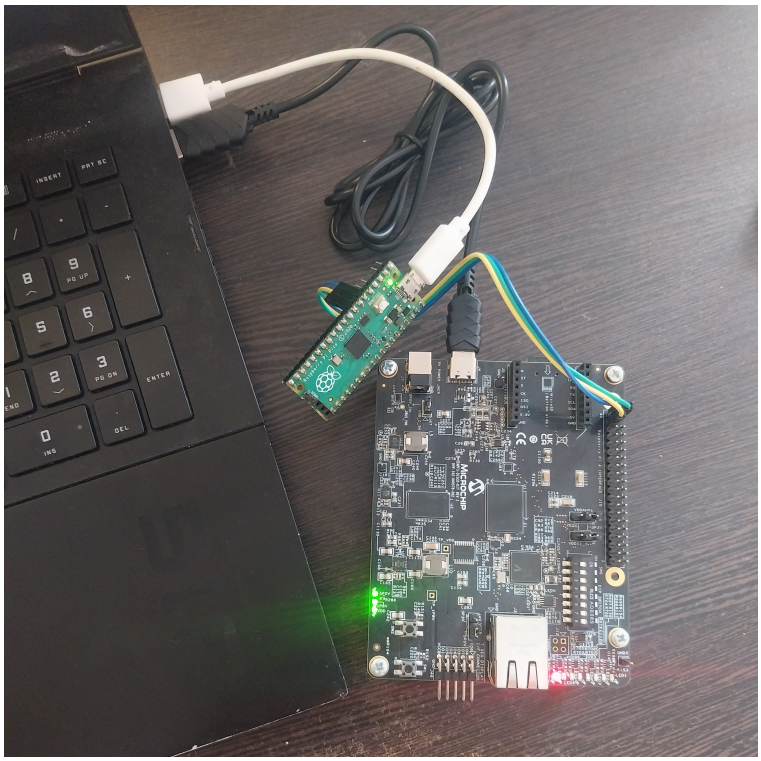

Figure 2.1: The testing setup.

The tools might or might not work on other versions, but the manual is written for the above. The reader is expected to have a working knowledge of Linux and Python, and to be able to install software and run commands in a terminal.

Not all of it is needed for all of it. Table 2.1 says what each piece buys; leave a row out and everything that does not depend on it still works.

Tool	Needed for
Python 3.10 or later	everything on the host
pyserial	the python driver, both front ends
PySide6	the graphical front end, `--gui`, only
USB-TTL adapter	reaching `libre_ila_uart` from the PC
Libero SoC	building a core into a design and programming the FPGA
SoftConsole	the baremetal C driver, and the demo application this chapter runs
GHDL	optional, runs the simulations under `tests/hdl/`
GTKWave	optional, opens the captured `.vcd`; any VCD viewer will do

Table 2.1: What each prerequisite is for.

2.1.1 Installing the host packages

Both python packages are in apt on Debian and Ubuntu:

```
sudo apt install python3-serial
sudo apt install python3-pyside6.qtwidgets python3-pyside6.qtgui
```

PySide6 ships one package per Qt module there, so `python3-pyside6` is not a package and `python3-pyside6.qtcore` on its own is not enough. Everything except `--gui` works without either of the two.

Ubuntu 22.04 predates PySide6's apt packaging, so on the tested system those two packages do not exist at all. Install it into a virtual environment instead, as in Section 2.1.2.

The optional tools are one line as well:

```
sudo apt install ghdl gtkwave
```

Neither is needed to build a core or take a capture. GHDL runs the simulations under `tests/hdl/`, and GTKWave is what the driver's `--waveform-viewer` defaults to when it opens a capture for you.

Finally, reading a serial port on Ubuntu needs membership of the `dialout` group. Add yourself once and log out and back in, otherwise every connection fails with a permission error on `/dev/ttyUSB0`:

```
sudo usermod -aG dialout $USER
```

2.1.2 Creating a Python virtual environment

This is how PySide6 is installed on a distribution that does not package it, and it is the tidier route on any distribution, since nothing lands system wide.

1. Install Python 3.10 or later, pip and the venv module:

   ```
   sudo apt install python3 python3-pip python3-venv
   ```

2. In the root of the repository, create a virtual environment named `venv-libreila`:

   ```
   python3 -m venv venv-libreila
   ```

3. Activate the virtual environment:

   ```
   source venv-libreila/bin/activate
   ```

4. Install the required packages:

   ```
   pip install pyserial PySide6
   ```

The name and the place matter. The root `Makefile` and the GUI test under `tests/python/05_gui/` both look for `venv-libreila` at the repository root and fall back to it when the system `python3` has no PySide6, so a virtual environment put anywhere else has to be activated by hand for every command in this manual.

To check that one of the two has it, without running anything else, from the repository root:

```
make check-pyside6
```

It prints an install line if neither the system `python3` nor the virtual environment has PySide6, and stays silent when one of them does.

2.2 Getting the sources

TODO: The clone, then a directory tour in one short table: `hdl/`, `codegen/`, `drivers/`, `tests/`, `docs/`.

2.2.1 Cloning the repository

Make a local clone of the repository,

```
    git clone https://github.com/yashparitkar/LibreILA
```

2.2.2 Importing the Libero project

1. Open Libero
2. In the Project tab → Open Project → Browse to {repo}/tests/full/libre_ila_uart/libre_ila_uart_libero/libre_
3. You can view the block design in `Design Heirarchy`, it contains the libre_ila, axi4smaster_pktgen (to generate dummy axistream traffic), a simple heartbeat ip and other MPFS blocks.

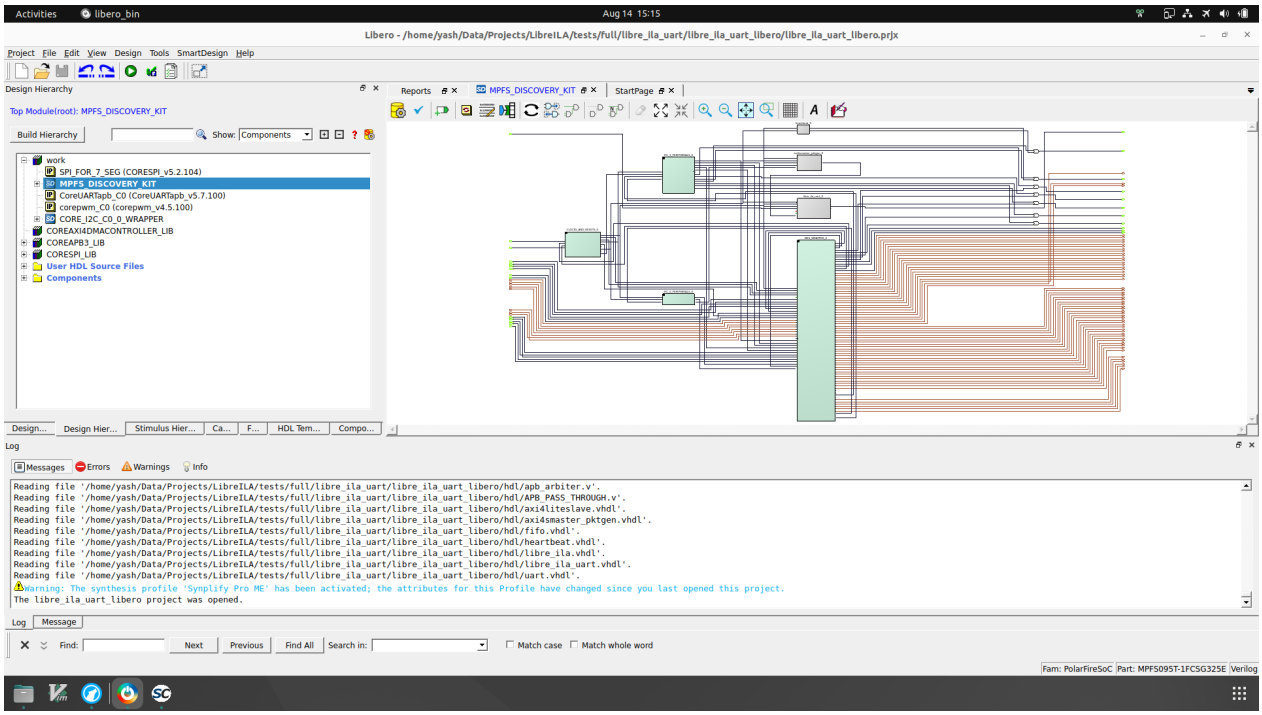

Figure 2.2: Exploring the block design.

4. Connect the Microchip PolarFire SoC development board to your PC via USB.
5. In the `Design Flow` tab, click `Run PROGRAM Action`, it should do all the synthesis, placement, routing and programming of the FPGA.

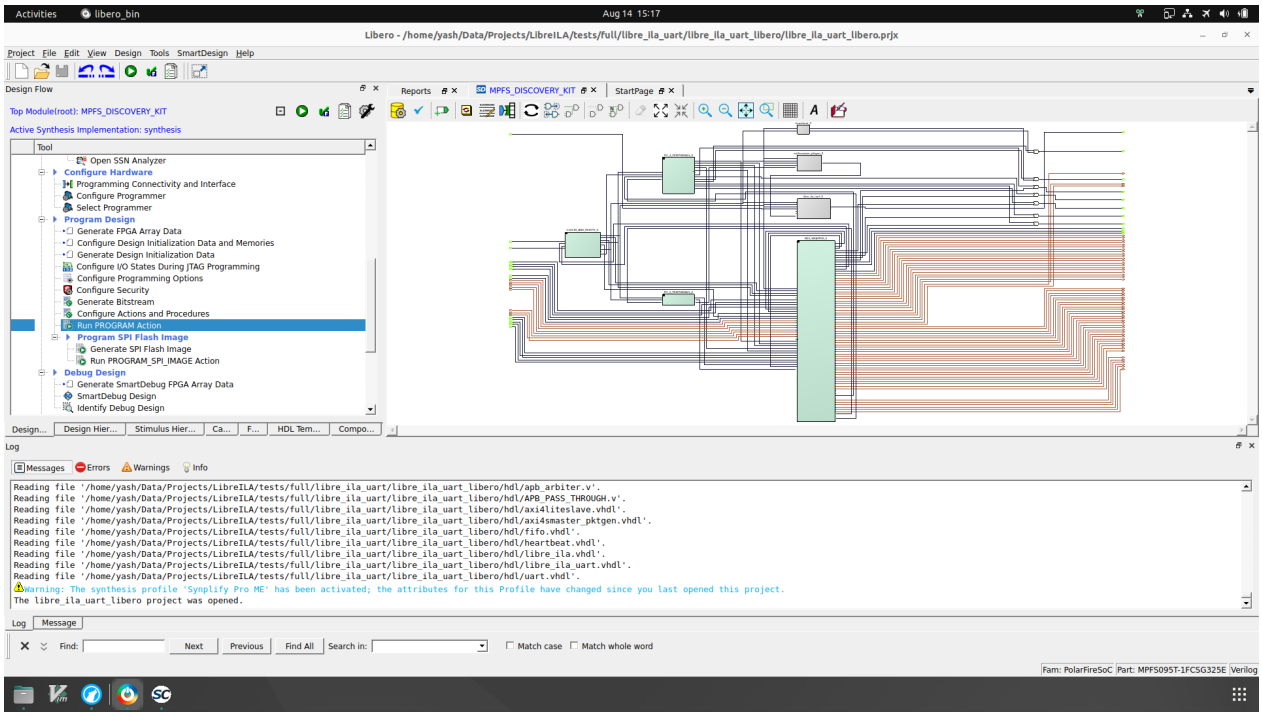

Figure 2.3: Run PROGRAM Action.

2.2.3 Importing the SoftConsole project

1. Open SoftConsole
2. Right click in the Project Explorer tab → Import → General → Existing Projects into Workspace → Next → Browse → Select {repo}/tests/full/libre_ila_uart/libre_ila_uart_sc → Finish

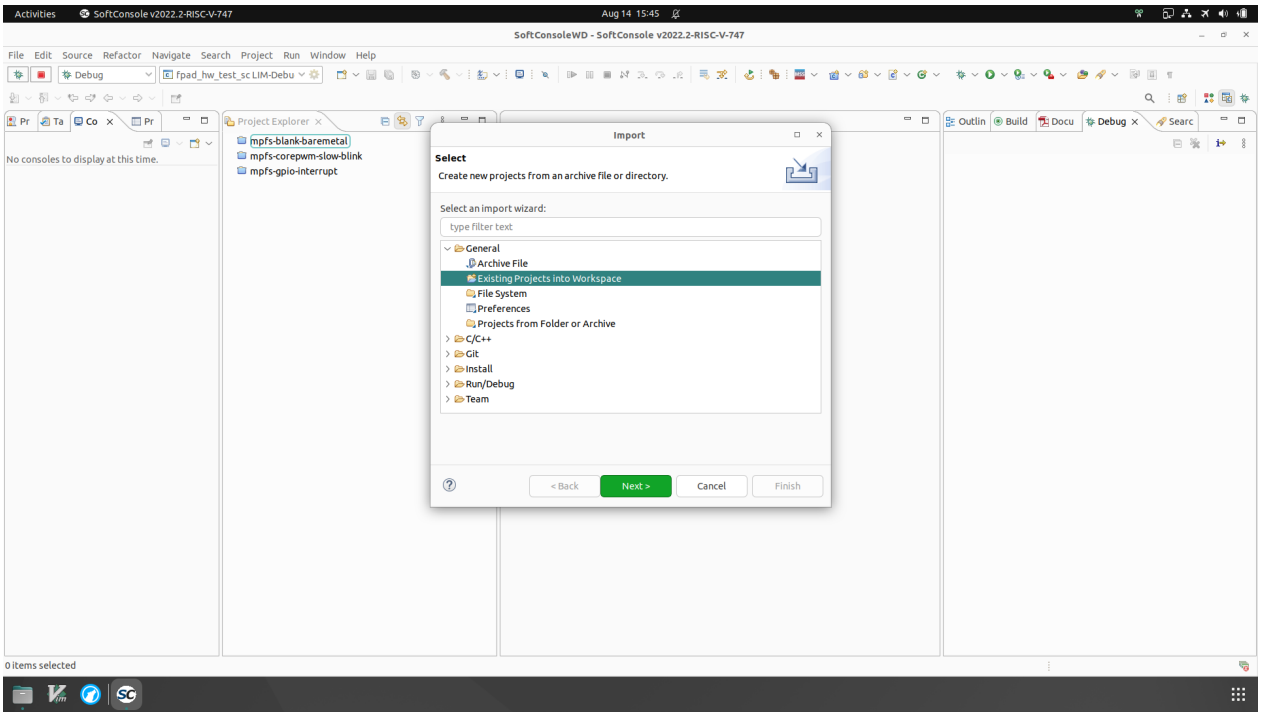

Figure 2.4: Importing the project.

3. You can view the source code in `src` folder, the file `src/application/hart1/u54_1.c` contains the main function which initializes the UART and sends some dummy data over AXI4Stream to the libre_ila core.

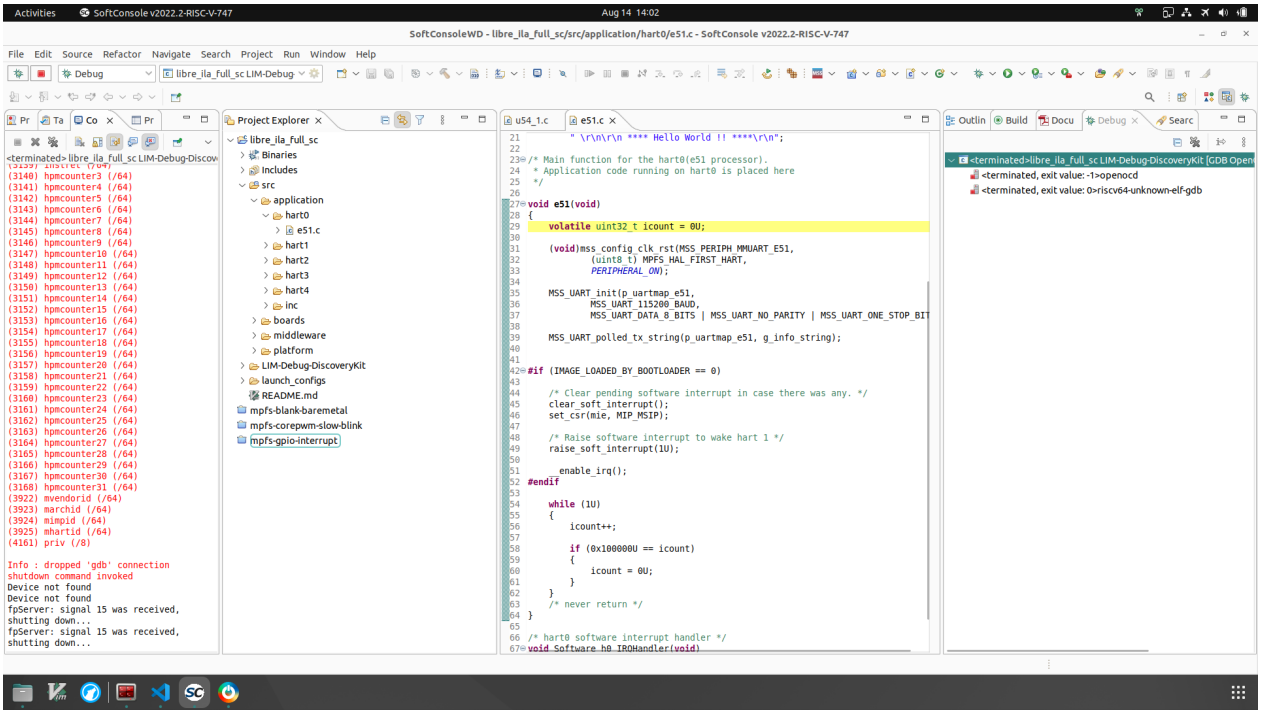

Figure 2.5: Opening the SoftConsole project.

4. Change the Build Configuration to LIM-Debug-DiscoveryKit

5. Build the project by clicking on the hammer icon in the toolbar.
6. Debug the project by clicking on the bug icon in the toolbar, it should program the SoC and start the application.

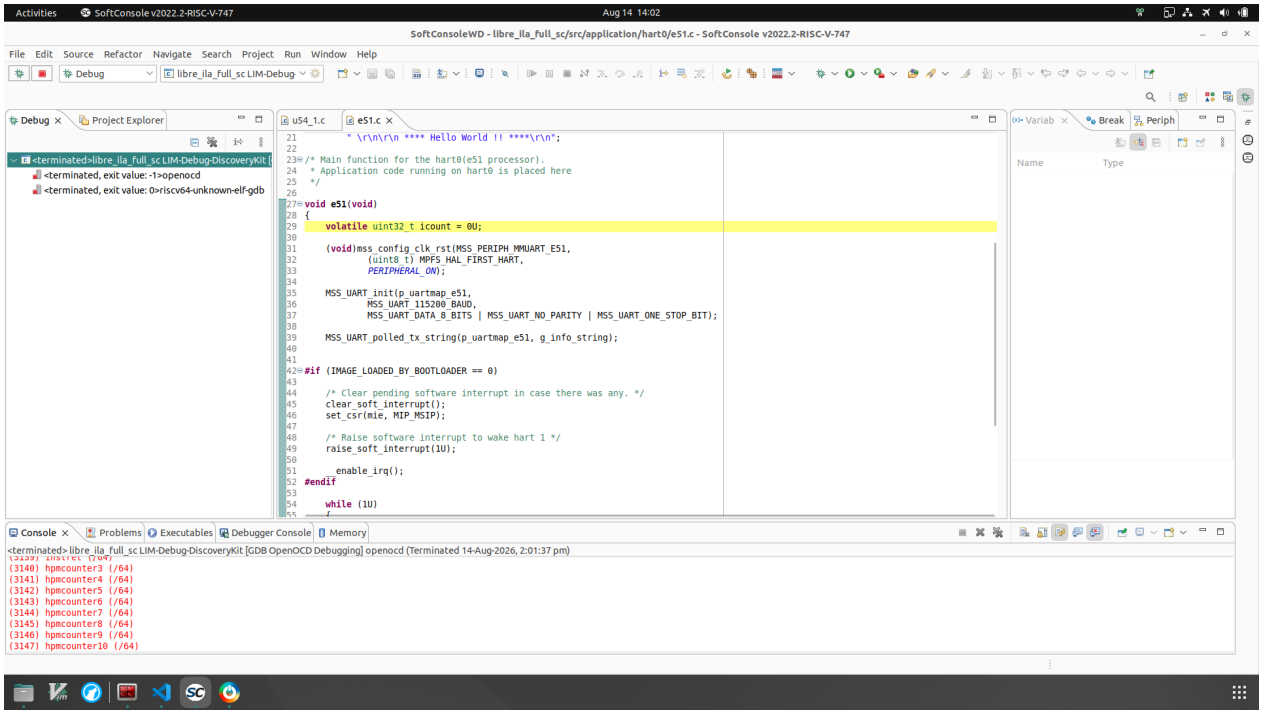

Figure 2.6: The Debug view.

7. Once the DDR training is complete, the application will stop at breakpoint in `src/application/hart0/u54_0.c` file, click on the Resume button in the toolbar to continue the execution of the application.

2.3 Your first capture

TODO: The whole loop on the stock 64-bit AXI4Stream build, every command shown and every output pasted: generate the core, add it to a project, build, connect the USB-TTL adapter, add the device, set a trigger, arm, wait, read back, open the VCD. The trigger is the datasheet's worked example, TVALID and TLAST rising, so the two documents agree.

1. Start the debug project on SoftConsole
2. Open a the serial terminal to connect with the MSS UART, you can use any serial terminal of your choice, we will use `minicom` here. Note that, if the drivers for the MPFS is installed properly, you will see three serial devices in `/dev/` directory, one of them will be streaming the output from the MSS UART. In our case, it is `/dev/ttyUSB2`.

```
minicom -D /dev/ttyUSB2 -b 115200
```

3. In the serial terminal, you should see the following output:

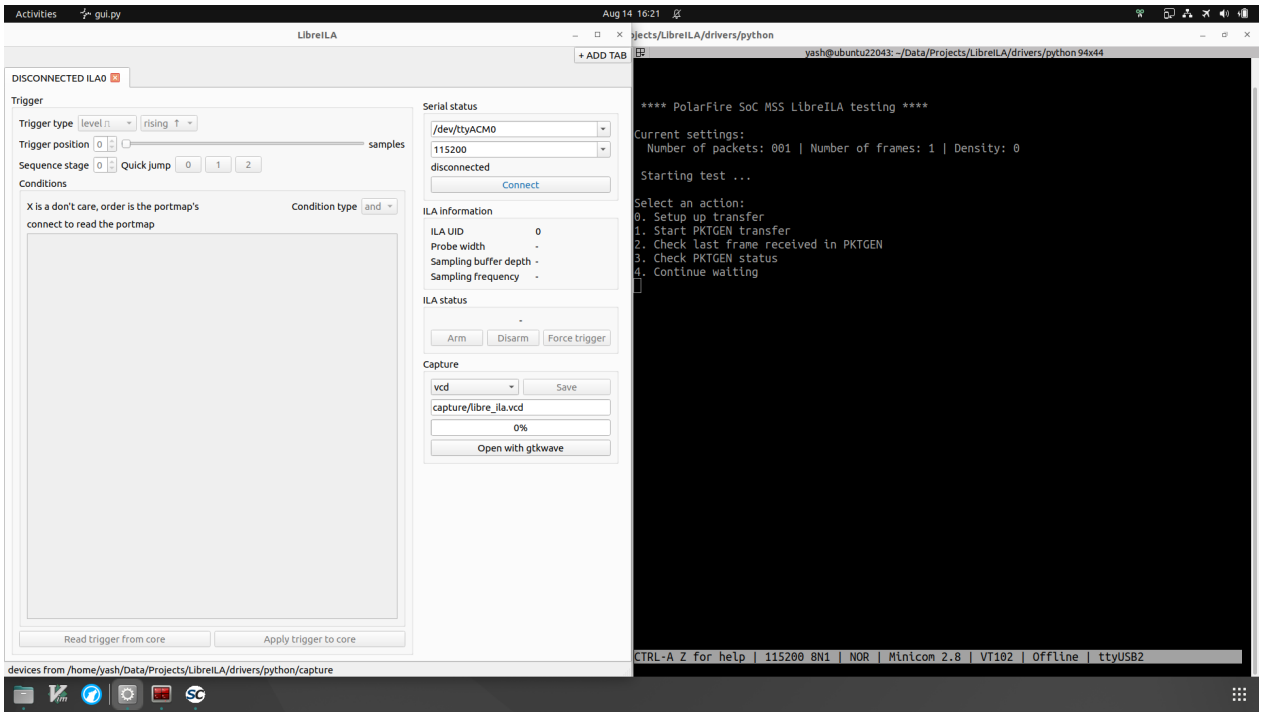

Figure 2.7: MSS UART connected.

4. Configure the pktgen using the menu options, we are using following configuration:

```
Packet size: 250 frames
Packet count: 250 packets
Packet density: bubble per 2 frames
```

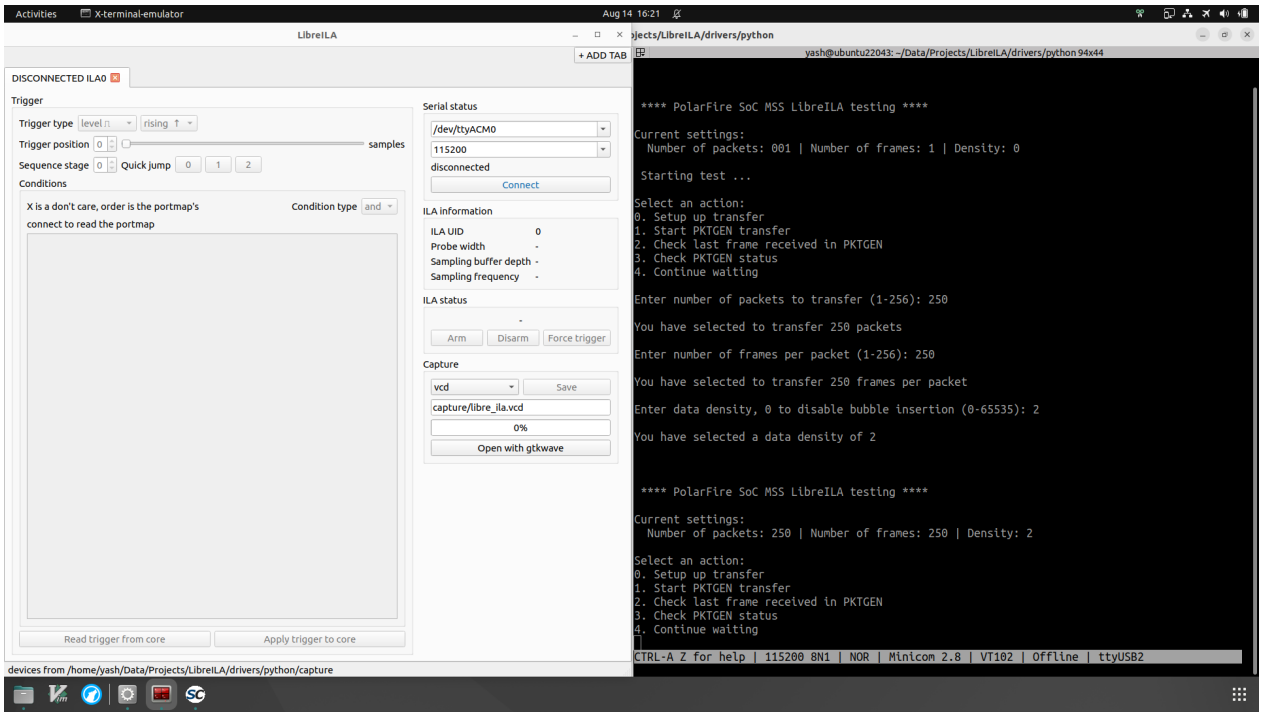

Figure 2.8: Configuring PKTGEN.

5. Open the GUI front end, you can use the following command to open the GUI:

```
python3 libreila.py --gui
```

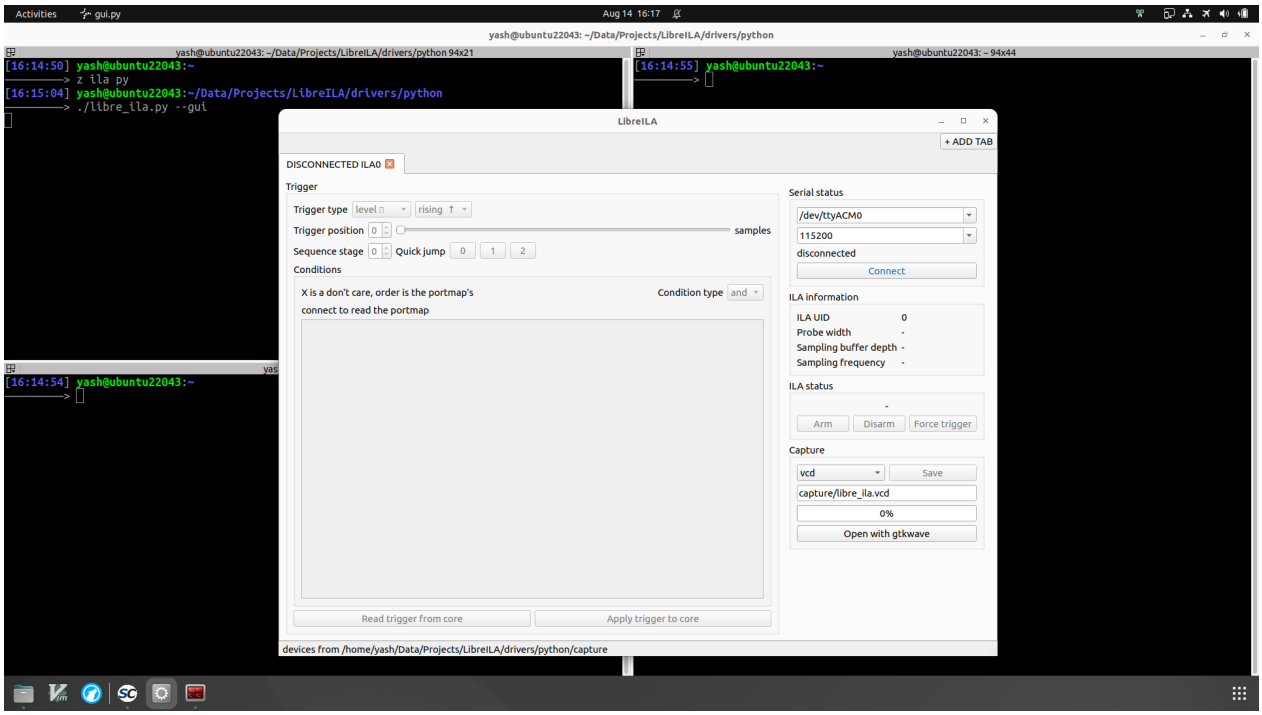

Figure 2.9: Launching LibreILA.

6. In the GUI, click on `Add Device` and select the serial port to which the USB-TTL adapter is connected, in our case it is `/dev/ttyACM0`. Click on `Connect` to connect to the device.
7. Set the trigger edge, rising and move the position to somewhere middle and trigger condition to `TVALID and TLAST rising`. Click on `Arm` to arm the trigger.

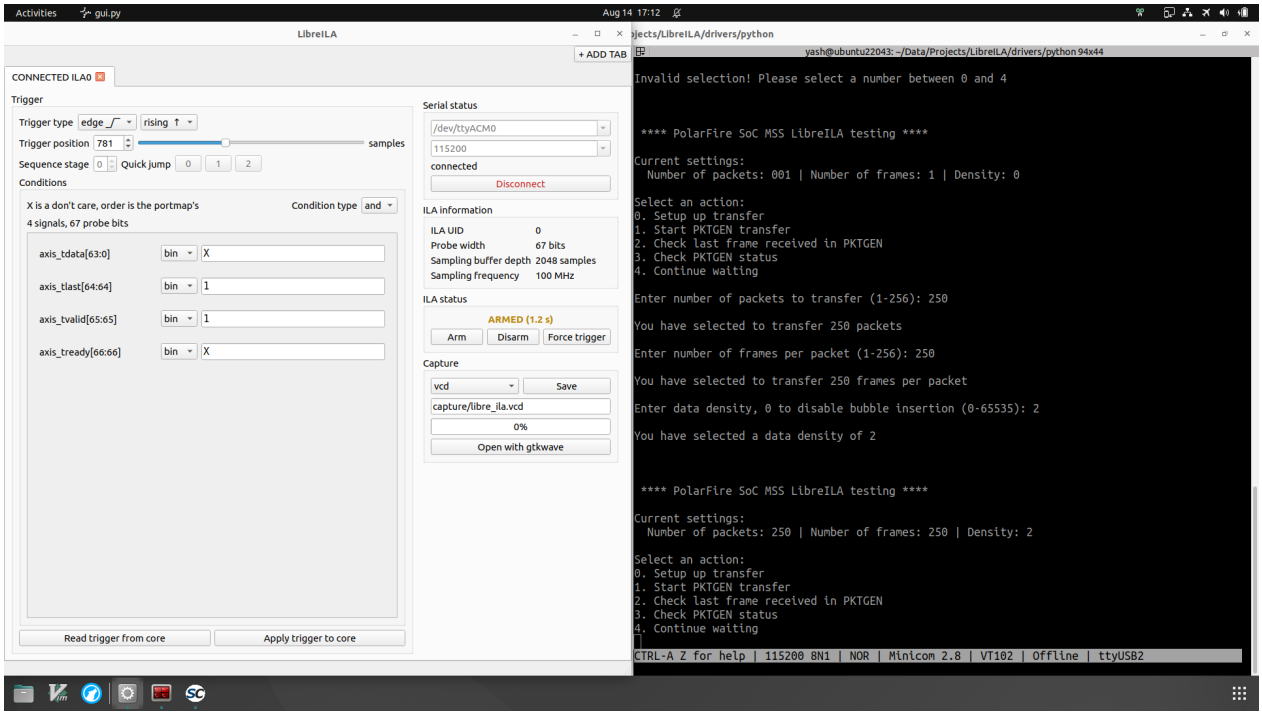

Figure 2.10: Configuring and arming the ILA.

8. Fire the packet generator by selection option `1. Start PKTGEN transfer` on the MSS serial terminal
9. The ILA should've captured the data and the ILA status will change to `Triggered` and then to `Done` quickly.

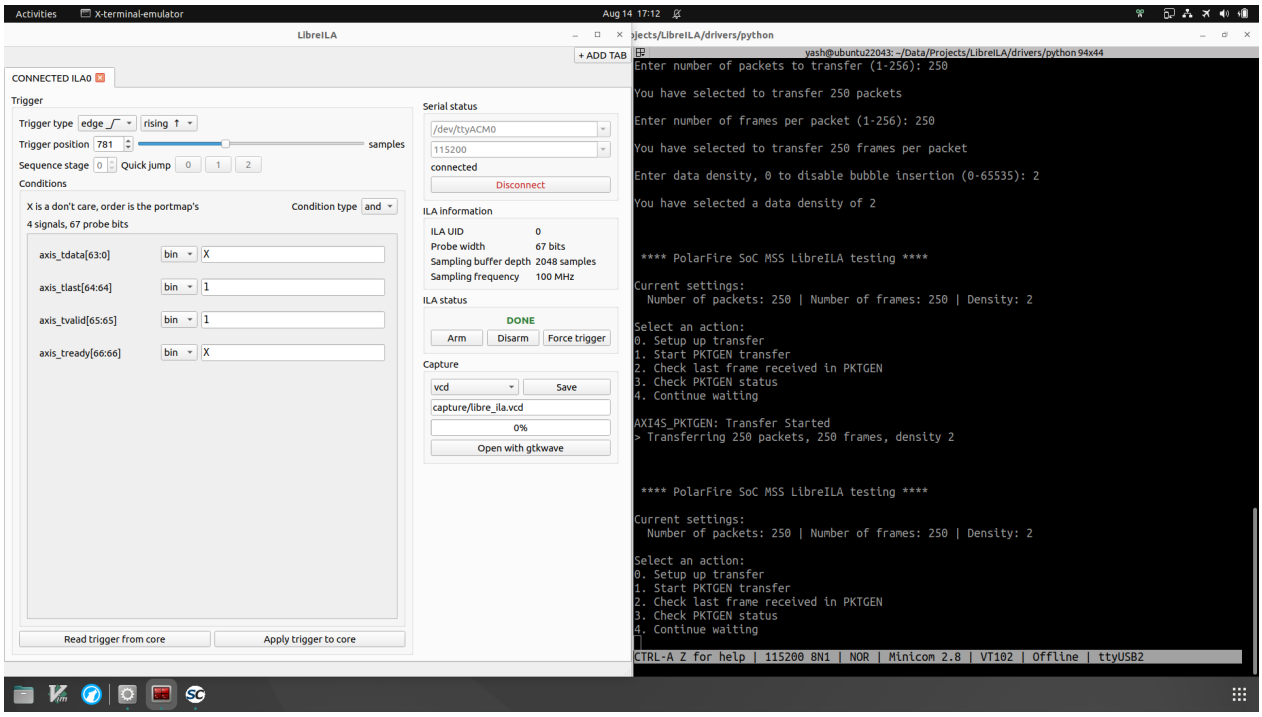

Figure 2.11: Firing PKTGEN, capture done.

10. Press `Save` in the `Capture` section to read data from the ILA and save it to a VCD file.
11. Press `Open with gtkwave` to open the capture in gtkwave
12. In gtkwave, right click on the `libre_ila` → Recure Import → Insert to add signals to waveform window
13. Tip: click on the "Magnifying glass with green tick" to fit the capture in the window

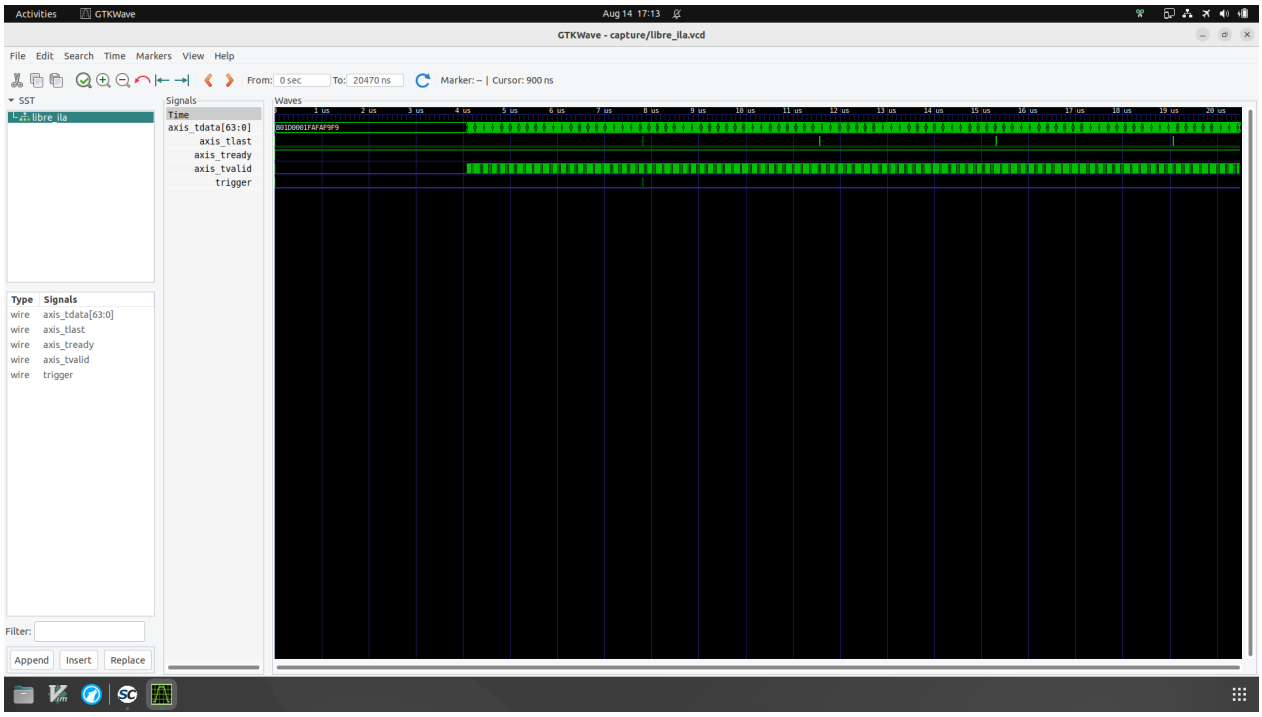

Figure 2.12: The GTKWave window.

2.4 What just happened

TODO: Half a page retracing the steps and naming the chapter that covers each in full: the map into the rest of the manual.

Chapter 3

The Code Generator

TODO: A build is fully described by `portmap.csv` and `configuration.csv`: how to write both, how to run the generator, and two recipes at the end.

3.1 Working of the code generator

TODO: Find and replace against the templates in `hdl/`, driven by the `^^XX` directives. The directive table (DI, DO, MI, MO, SH, MX), and why a directive listed for a file but not found aborts the run: a dropped block cannot become an unconnected port.

3.2 Writing configuration.csv

TODO: One line per generic, as `generic,type,value`. The generics that matter in practice: GEN_TYPE, which picks core only or core plus wrapper, G_SAMP_BUFF_DEPTH, G_EXTERNAL_TRIG and the UART settings. G_UID may be omitted entirely and may be written in hex. The full ranges stay in the datasheet.

3.3 Writing portmap.csv

TODO: Three columns, `signal,width,type`, listed LSB first, and that order is the probe bit order everything else inherits. Parsing is strict: no whitespace around fields, comments only in column 0. The stock AXI4S portmap and one other example.

3.3.1 Choosing the signal type

TODO: `in`, `out` and `mon` as a table, described from the `probe_slave_` side.

3.3.2 Using bidirectional signals

TODO: A signal assignment cannot pass a bidirectional line through, so the splice goes on the fabric side of the IO buffer, across the three signals the tristate is built from. The QPI flash example, including the turnaround it captures and a pad probe cannot. Where a bus cannot be spliced at all, `mon`.

3.4 Running the generator

TODO: The command lines and the flag table. Output lands in `gen_axis/` for the stock build and `gen/` otherwise, overridden by `--outdir`, and neither is tracked. The trap worth calling

out: after editing anything in `hdl/`, `gen_axis/` has to go, by hand or with `make clean`, or the tests keep simulating the previous build.

3.5 Building a generic logic analyzer

TODO: Two ways there: every port `mon`, or every port `in` with the master side left open. Which one to prefer and why, and that a `mon` build has no pass-through, so the core only observes.

3.6 Probing multiple channels in same build

TODO: Several channels on one ILA, as long as they share a clock.

Chapter 4

Integrating LibreILA into a Design

TODO: What to do with the generated code once it exists.

4.1 Files to add

TODO: What the generator emits for GEN_TYPE 0 and 1, and which of those files go into the project. VHDL Style Guide can be run over them first where a project has a house style.

4.2 Core or wrapper

TODO: Choosing between the two, and what each one costs and buys.

4.3 Splicing the probe into a link

TODO: What the probe master and probe slave sides are, and where the core sits in the link.

4.4 Clocks and reset

TODO: `samp_aclk` takes the clock the probed link runs on, not a convenient free-running one; `s_axil_aclk` takes whatever the bus runs on. The practical reset rule, with the requirements themselves left in the datasheet.

4.5 Timing constraints

TODO: Work in progress: not verified in SDC yet.

4.6 Connecting the UART

TODO: Any two pins will do, chosen in the design constraints.

4.7 Checking the build

TODO: Confirming the sample buffer was inferred as RAM and that timing is met.

Chapter 5

The Software Stack

TODO: The layers the python driver is built from, and what each one owns.

5.1 Architecture

TODO: The architecture diagram, and the division of labour between the layers.

5.2 The session

TODO: What a session is, who uses it, and the rules for using one.

5.3 The register driver

TODO: How the UART carries AXI4Lite register access, and the link's fail safes: timeout and resync.

5.4 The trigger translator

TODO: Per-signal patterns such as `0b00101xxxx`, translated to and from the condition/mask pair.

5.5 The VCD writer

TODO: Reads `portmap.csv`, checks the widths add up to the probe width the core reports, unrolls the circular buffer and writes named signals.

Chapter 6

The Command Line Interface

TODO: Every command with its usage and a worked scenario. The transcripts come from real runs against hardware, not from plausible-looking output written by hand.

6.1 Telling the tool about a core

TODO: `--add-device` connects, reads the UID back and writes the device store; after that a core is addressed by `--device`. The point to make loudly: `--device` selects by *UID*, not by the order devices were added, and a core built without a G_UID reports 0. `--device-dir` has to agree between the CLI and the GUI.

6.2 The verbs and the order they run in

TODO: The flags are verbs, and they are applied in a fixed order whatever order they are given in. The ordering table, then a whole capture as one line, and the same capture split across invocations: nothing about the session is kept on the host.

6.3 Configuring the trigger

TODO: Condition and mask as one number covering the whole probe word, in any base, with bit 0 the first signal in the portmap. These do *not* use the portmap names. Type and reduction, omitted fields keeping what the core already had, and a value with bits above the probe width being refused rather than trimmed.

6.4 Worked scenarios

TODO: Three or four end-to-end runs with real pasted output: trigger on a condition, force a trigger that never came, disarm and retry, and a capture split across two invocations.

6.5 Debugging the link

TODO: `--debug` logs every UART transaction. A real log extract and how to read it: the byte counts matter as much as the errors, because a reply that is short or long for its request means the two ends disagree about which request is being served.

6.6 Exit statuses

TODO: The exit codes, a sentence each on what actually causes them, and a pointer to the troubleshooting chapter.

Chapter 7

The Graphical Interface

TODO: Screenshots carry this chapter. `make shots` in `tests/python/05_gui` renders the real GUI against two fake cores, so every screenshot here can be regenerated without hardware, in one pass, once the shot list is settled.

7.1 Starting it

TODO: `--gui`, what it needs installed, and that it reads the same device store as the CLI, so `--device-dir` must agree.

7.2 Tabs and connections

TODO: One tab per stored device, titled by connection state and UID. A tab owns one session for as long as it lives, so the port is opened once and held. A tab starts disconnected: opening the window opens no ports.

7.3 Running a capture

TODO: The workflow, a screenshot per step: connect, review the trigger, arm, watch the state, force or disarm if needed, capture with the progress bar, open in the waveform viewer.

7.4 What it does not do yet

TODO: The trigger view is read only in this pass. The per-signal condition table is next, and the translator behind it already exists; saying so here saves a reader hunting for a control that is not there.

Chapter 8

The Baremetal C Driver

TODO: The path for `libre_ila` on AXI4Lite, which the manual does not cover at all today. A reader on this path never touches the python driver, so the chapter has to stand on its own.

8.1 Adding the driver to a project

TODO: The three files and what each is for. The AXI4Lite accessors are HAL calls, so which toolchain the shipped version assumes has to be stated plainly.

8.2 Definitions you must supply

TODO: The three CORE_LIBRE_ILA_ defines size the arrays the caller declares and nothing else; the driver reads none of them. Why array sizes cannot come from the hardware, and why the length travels alongside the pointer, so a mismatch fails at the call rather than overrunning.

8.3 Initialising

TODO: `LIBRE_ILA_init()` reads the probe width, buffer depth and sampling clock out of the core and derives the register map, so re-synthesising with a different probe width needs no firmware rebuild. It also picks up the UID.

8.4 Configuring the trigger

TODO: The probe is opaque: one flat word, bit 0 upwards. The pattern of naming your own bits with a define and setting them through the word/mask macros, then the configure call with its reduction mode.

8.5 Running a capture

TODO: Position, arm, wait or poll status, force trigger, disarm, read back, as one complete compiling example rather than fragments.

8.6 Several cores from one binary

TODO: Each instance carries its own geometry, so one binary can drive cores of different probe widths. Their arrays are sized with the width macros and indexed with the sample macros,

passing that instance's lane count. The magic key says a core is there, the UID says which one.

8.7 Porting to another toolchain

TODO: The register accessors are the only tool-dependent part: exactly which functions to reimplement and what they must do, so a user on a non-Microchip toolchain gets a checklist rather than a warning.

Chapter 9

Reading the Capture

TODO: What the user ends up with, and how to read it. A short chapter, but the one that turns a file into an answer.

9.1 The VCD file

TODO: Signals named from `portmap.csv`. Delta encoded: the first sample states every signal, after that only what moved. The width check that refuses a readout when the portmap has drifted from the synthesised core, and why that check exists.

9.2 The trigger marker

TODO: VCD has no marker of its own, so the trigger is carried as an extra one-bit wire that pulses for exactly the sample it fired on.

9.3 The time axis

TODO: Timestamps in picoseconds, one sample apart at the sampling clock the core reports. The rate is G_SAMP_CLK_FREQ as synthesised, so a wrong generic gives a correct capture on a wrong time axis.

9.4 In the waveform viewer

TODO: Opening the file, finding the trigger, and a screenshot of a real capture. `--waveform-viewer` for a reader who does not use GTKWave.

9.5 Reading the raw buffer yourself

TODO: For the baremetal path or a custom host: unrolling from the first index, the trigger index and the modulo arithmetic, with the register offsets left to the datasheet capture procedure.

Chapter 10

Troubleshooting

TODO: The chapter that most directly stops a user hunting through READMEs. Every entry is a symptom the user actually sees, the cause, and the fix: a summary table first, then a subsection per group with the detail.

10.1 Generation and build

TODO: Stale `gen_axis/` after editing `hdl/`. Portmap rejected for whitespace around fields. Directive not found aborting the run. Sample buffer inferred as registers and the area blowing up. Timing failing between the two clock domains because they were never declared asynchronous.

10.2 The link

TODO: No port, busy port, timeout, desynced reply. A baud mismatch between G_BAUD_RATE and `--baud`. RX and TX not crossed over. How `--debug` tells these apart, and what the resync line means.

10.3 The trigger

TODO: Fires immediately on arming: a level trigger whose condition already held, which is the single most common surprise. Never fires: mask, reduction, or a condition that cannot occur; recover with force trigger. An all-zero mask in AND mode matching vacuously. Mask bits set in the padding above the probe width.

10.4 The capture

TODO: Signals named wrongly or shifted: portmap drifted from the synthesised core. Buffer looks like noise before the window start: reading without unrolling from the first index. Time axis wrong: G_SAMP_CLK_FREQ does not match the real clock. Talking to the wrong core: `--device` is a UID, not an index.

Appendix A

Simulations

TODO: What the simulations cover, and how to run them with no hardware attached.

A.1 The test suite

TODO: `make sim` and what each group needs: the HDL simulations need GHDL and a generated core, the python and C tests need neither. What each group actually proves.